TECHNICAL DOCUMENTATION

StudyHub

A peer-to-peer learning platform for university students — study groups, shared notes, a Q&A forum, tutor booking and Mobile Money payments.

Document type
Engineering handover

Version
2.0

Author
Tazanu Stanley

Repository
github.com/Tazanu/studyhubwebapp

Live site
studyhubwebapp.vercel.app

Generated
11 September 2026

— Contents

This document describes how StudyHub is built: the architecture, every third-party tool and what it does, the data model, the security and payment mechanics, and how to run and deploy the system. It is written for a developer taking the project over.

1 System overview

StudyHub is a web application that lets university students help each other study. A signed-in student can join subject study groups and chat in them in real time, upload and download course notes, ask and answer questions on a forum, and book a peer tutor for a paid one-to-one session. Tutors apply, are reviewed by an administrator, set their own rate and availability, and are paid through Mobile Money.

It is a two-tier system: a React single-page application in the browser, and an Express REST API with a PostgreSQL database. They are deployed independently and communicate over HTTPS with JSON, plus a WebSocket connection for chat.

Scale

Frontend source	~14,700 lines across 26 pages, 17 shared UI primitives, 6 custom hooks
Backend source	~6,000 lines across 12 route modules, 7 services, 7 middleware
Database	25 tables, 7 migrations, 476-line Prisma schema
API surface	88 REST endpoints
Languages	English and French, 1,228 translation keys each

The four product areas

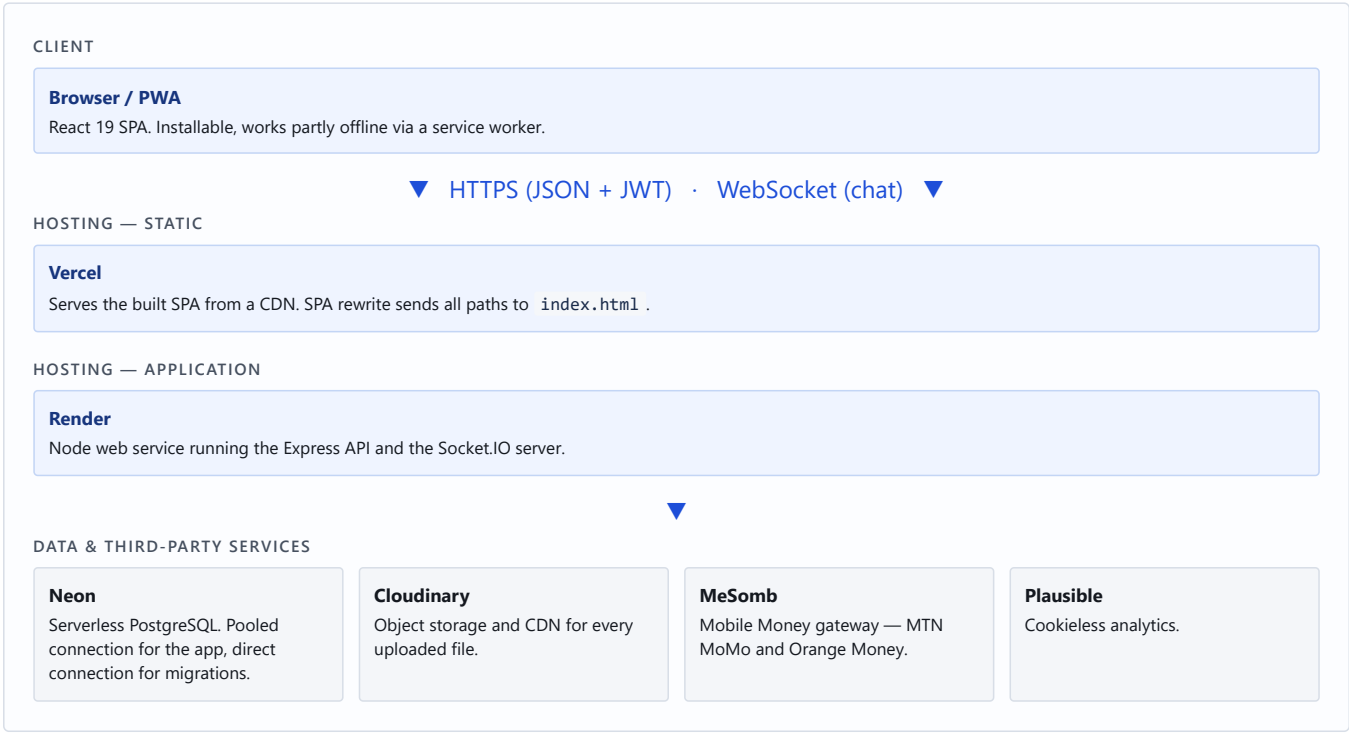
AREA	WHAT A USER CAN DO	KEY TABLES
Study groups	Create or join a group (open or approval-gated), chat in real time with file attachments, replies, edits and emoji reactions, manage members as an owner	`groups`, `user_groups`, `group_messages`, `group_join_requests`
Notes	Upload course notes with tags and a subject, browse and filter, preview PDFs and images in-app, download. Notes may be free or paid	`notes`, `premium_notes`, `purchased_notes`
Q&A forum	Ask a question with optional voice recording and images, answer, vote, accept a best answer, bookmark, follow. Reputation accrues from activity	`questions`, `answers`, `question_votes`, `answer_votes`
Tutoring	Apply as a tutor, set rate and weekly availability, be approved by an admin; students browse, book a slot and pay by Mobile Money	`tutors`, `tutor_availability`, `bookings`, `session_credits`

User roles

ROLE	CAPABILITIES
Student	The default on registration. Full access to groups, notes, Q&A; can buy premium notes and book tutors.
Tutor	A student whose application has been approved. Additionally: appears in the tutor directory, manages availability, accepts or declines bookings, may publish paid notes.
Admin	Reviews tutor applications and paid-note submissions, manages users (ban, promote), sees platform statistics and payment diagnostics.

2 Architecture

Three managed services, each free-tier, each doing one job. Nothing is self-hosted.



Why the frontend and backend are hosted separately

The frontend is static files and benefits from a CDN; the backend is a long-lived Node process that must hold WebSocket connections and run a background timer. Those have different runtime needs, so they are deployed to different platforms. The cost is that they sit on different origins, which makes CORS configuration a first-class concern (section 11) and means the API URL must be baked into the frontend at build time (section 13).

Request flow — a typical authenticated call

1. The browser calls `api.get('/notes')`. An Axios request interceptor attaches `Authorization: Bearer <token>` from `localStorage`.
2. Render's proxy terminates TLS and forwards to Express. `trust proxy` is set so the real protocol and client IP survive.
3. Express redirects to HTTPS if `x-forwarded-proto` is not `https`, then Helmet applies security headers.
4. CORS checks the `Origin` against an allow-list. A rejected origin is logged by name.
5. The rate limiter for that path runs.
6. The route's `authenticate` middleware verifies the JWT and sets `req.userId`.
7. The handler queries PostgreSQL through Prisma and returns JSON.
8. A response interceptor in the browser catches `401` (clear session, redirect to login) and retries transport failures (section 4).

Directory layout

```
studyhub-v2/
├── frontend/
│   ├── src/
│   │   ├── pages/          26 route components
│   │   ├── components/
│   │   │   ├── ui/         17 shared primitives (Button, Modal, Badge…)
│   │   │   ├── home/       marketing sections
│   │   │   └── tutor/      tutor-profile sections
│   │   ├── context/        AuthContext, ThemeContext
│   │   ├── hooks/          6 custom hooks
│   │   ├── lib/            formatting, tiers, analytics, helpers
│   │   ├── i18n/           config + en.json / fr.json
│   │   └── api/client.js   Axios instance + interceptors
│   ├── scripts/            check-i18n.mjs
│   ├── vite.config.js      build, PWA, sitemap/robots generation
│   └── vercel.json         SPA rewrite + cache headers
└── backend/
    ├── prisma/schema.prisma
    └── src/
        ├── routes/         12 modules → /api/*
        ├── services/       7 domain services
        ├── middleware/     auth, roles, rate limits, uploads, honeypot
        ├── socket.js       Socket.IO server
        └── index.js        app assembly and startup
```

3 Technology stack

Every third-party dependency, what it does in this project, and why it is there.

Frontend — runtime dependencies

PACKAGE	VERSION	ROLE IN STUDYHUB
`react` `react-dom`	19.2	UI framework. React 19 is used specifically for its native `<title>` hoisting in the SEO component.
`react-router-dom`	7.18	Client-side routing for 26 routes, including the `<Protected>` wrapper that gates authenticated pages.
`axios`	1.18	HTTP client. Configured once in `api/client.js` with token injection, a 45s timeout and cold-start retry.
`tailwindcss` `@tailwindcss/vite`	4.3	Styling. Tailwind v4's `@theme inline` exposes CSS custom properties as utilities that stay reactive to the runtime light/dark switch.
`framer-motion`	12.40	Animation — page transitions, the sliding active-tab pill, card hover lifts, list stagger.
`lucide-react`	1.21	Icon set. Replaced emoji icons throughout so icons inherit colour and scale.
`socket.io-client`	4.7	WebSocket client for group chat, wrapped in the `useGroupSocket` hook.
`recharts`	3.8	The weekly activity bar chart on the dashboard.
`react-pdf`	10.4	In-app PDF preview so a note can be read without downloading it.
`sonner`	2.0	Toast notifications for success and error feedback.
`react-helmet-async`	3.0	Per-page document head on the tutor profile, including JSON-LD structured data.

`i18next`
`react-i18next`
`i18next-browser-languagedetector`
	26.4
	17.0
	8.2
		Internationalisation: translation lookup, React bindings, and browser-language detection with a persisted override (section 12).

Frontend — build and tooling

PACKAGE	VERSION	ROLE IN STUDYHUB
`vite`	8.0	Dev server and production bundler. Also the place where `VITE_*` variables are inlined at build time.
`@vitejs/plugin-react`	6.0	JSX transform and Fast Refresh.
`vite-plugin-pwa`	1.3	Generates the web app manifest and a Workbox service worker with per-route caching strategies (section 4).
`eslint + react-hooks + react-refresh`	10.3	Static analysis. Caught two real bugs during the i18n work: a variable shadowing the translation function, and `t` used outside its component.
`sharp`	0.35	Build-time image compression for the social preview card and PWA icons.

Backend — runtime dependencies

PACKAGE	VERSION	ROLE IN STUDYHUB
`express`	5.2	HTTP framework. Express 5 propagates async handler rejections to the error middleware automatically.
`prisma` `@prisma/client`	5.22	ORM and migration tool. The schema is the single source of truth for all 25 tables; the generated client gives type-aware queries.
`pg`	8.22	PostgreSQL driver used underneath Prisma.
`jsonwebtoken`	9.0	Signs and verifies the stateless session token.

PACKAGE	VERSION	ROLE IN STUDYHUB
`bcryptjs`	`3.0`	Password hashing at 12 rounds.
`helmet`	`8.3`	Security response headers including a Content-Security-Policy and HSTS.
`cors`	`2.8`	Cross-origin access control with an explicit allow-list — required because the SPA is on a different origin.
`express-rate-limit`	`8.5`	Five separate limiters: general API, auth, login, payment initiation, payment polling.
`multer` `multer-storage-cloudinary`	`2.2` `4.0`	Multipart upload parsing, streaming straight to Cloudinary rather than to local disk.
`cloudinary`	`1.41`	Storage SDK. Also used to build signed, expiring URLs for paid files.
`socket.io`	`4.7`	WebSocket server with JWT handshake authentication and per-group rooms.
`@hachther/mesomb`	`2.0`	Mobile Money SDK for MTN MoMo and Orange Money in Cameroon.
`axios`	`1.18`	Server-side HTTP, used for direct provider calls where the SDK is insufficient.
`dotenv`	`17.4`	Loads `.env` in local development.
`nodemon`	`3.1`	Dev-only auto-restart.

Infrastructure and external services

SERVICE	USED FOR	NOTES
Vercel	Frontend hosting	Global CDN, automatic deploys from the `master` branch, SPA rewrite so client routes resolve.
Render	Backend hosting	Node web service. On the free tier it sleeps after ~15 minutes idle — the frontend compensates (section 4).
Neon	PostgreSQL	Serverless Postgres. Requires two URLs: a pooled one for the app and a direct one for migrations (section 6).
Cloudinary	File storage + CDN	Every avatar, note, chat attachment, voice recording and proof document.
MeSomb	Payments	Cameroonian Mobile Money aggregator. Asynchronous collect with polling.
Plausible	Analytics	Chosen because it is cookieless — no consent banner is legally required, and none is shown as a gate.
GitHub	Source control	Single repository holding both apps; both hosts deploy from it.

4 Frontend

Routing

All 26 routes are declared in `App.jsx`. Authenticated routes are wrapped in `<Protected>`, which redirects to `/login` when no valid session exists. A catch-all `*` route renders the custom 404 page.

ACCESS	ROUTES
Public	`/` `/about` `/login` `/register` `/contact` `/terms` `/privacy` `/cookies`
Authenticated	`/dashboard` `/profile` `/profile/:id` `/groups` `/groups/:id/chat` `/notes` `/notes/:id` `/qa` `/qa/ask` `/qa/:id` `/settings` `/premium` `/tutors` `/tutor/:id` `/become-tutor` `/tutor-dashboard`
Admin only	`/admin`

State management

There is no Redux or equivalent. State is deliberately kept local, with two React contexts for the genuinely global concerns:

- `AuthContext` — the current user, the token, and `login` / `logout` / `refreshUser`. Persisted to `localStorage` so a reload does not sign the user out.
- `ThemeContext` — light or dark, applied as a `data-theme` attribute on the document root.

Everything else is component state plus data fetched on mount. Several list pages poll on an interval (notes and groups every 15s, the Q&A forum every 20s) so other users' activity appears without a manual refresh.

Design system

The interface is built on a token-based design system rather than ad hoc styling. `index.css` defines a brand scale, neutral surfaces, semantic status colours, a shadow scale and radii as CSS custom properties; Tailwind v4's `@theme inline` exposes them as utility classes that stay bound to `var()` rather than being compiled to fixed values. That is what allows the light/dark toggle to work instantly with no rebuild.

Seventeen shared primitives in `components/ui/` — `Button`, `Card`, `Modal`, `Badge`, `Field`, `Input`, `Select`, `Textarea`, `EmptyState`, `Skeleton`, `StarRating`, `Stepper`, `Banner`, `ConfirmDialog`, `InlineConfirm`, `UserAvatar`, `HoneypotField` — replaced what had been the same visual component hand-rolled separately on each page.

> **DESIGN NOTE**
>
> `Badge` maps its `tone` prop to full literal class names rather than building them by string interpolation. Tailwind's scanner is static: a class assembled at runtime is never emitted into the stylesheet, so the interpolated version would have produced unstyled badges in production while looking fine in development.

Cold-start handling

Render's free tier sleeps the backend after roughly fifteen minutes of inactivity, and the first request that reaches a sleeping container is usually dropped while it boots. Without handling, a visitor arriving from a shared link sees a failure on a perfectly healthy service.

The Axios layer therefore retries — but narrowly. Only requests that never received a response at all are retried (up to twice, four seconds apart). Anything the server actually answered, including a 500, is left alone, because repeating a POST the server did process could duplicate it. While a retry is in flight the client emits an event that renders a "waking the server" banner, so the wait is explained rather than silent.

Progressive Web App

The app installs to a phone home screen and keeps working partly offline. The service worker is generated by `vite-plugin-pwa` with deliberately different strategies per route:

API PATH	STRATEGY	REASON
`/api/stats`	StaleWhileRevalidate	Landing-page counters; stale for five minutes is harmless.

API PATH	STRATEGY	REASON

`/api/groups`
`/api/notes`
`/api/questions`
	NetworkFirst (5s)	Fresh when online, readable offline. 24-hour cache.

`/api/tutors`	NetworkOnly	Rates and availability must never be served stale.

`/api/auth`
`/api/payments`
`*/messages`
`notes/*/download`
	NetworkOnly	Authentication, money, live chat and download counters must not be cached.

SEO artefacts generated at build

A custom Vite plugin emits `sitemap.xml` and `robots.txt` during the build, using the real deployed origin from `VITE_SITE_URL`. Sitemap URLs must be absolute — a relative one is silently ignored by crawlers — so generating them at build time avoids hardcoding a guess. Authenticated route prefixes are disallowed in robots.txt, since a crawler following them only ever reaches the login redirect.

5 Backend

Middleware chain

Order matters and is fixed in `src/index.js` :

#	MIDDLEWARE	PURPOSE
1	`trust proxy`	Makes the real client IP and protocol visible behind Render's reverse proxy. Rate limiting and the HTTPS redirect both depend on it.
2	HTTPS redirect	In production only. Redirects with **308** rather than 301 so the method and body of a non-GET request survive.
3	`helmet`	CSP, HSTS, nosniff, frame-ancestors none, and related headers.
4	`cors`	Explicit origin allow-list with credentials enabled. A wildcard is not permitted alongside credentials.
5	`express.json`	Body parsing, capped at 1 MB.
6	Static `/uploads`	Serves legacy local files, but refuses anything under `protected/` with a 403.
7	Rate limiters	Path-scoped. Polling limiter is registered before the strict payment limiter so it takes precedence on status routes.
8	Routes	Twelve modules mounted under `/api/*` .
9	`/api` 404	Unknown API paths return a clear 404 instead of falling through to a confusing 500.
10	Error handler	Logs fully server-side; returns a generic message. Never leaks a stack trace.

Route modules

MOUNT	ROUTES	RESPONSIBILITY
`/api/auth`	4	Registration, login, current user, password change.
`/api/groups`	21	Group CRUD, membership, join requests, messages, reactions, typing, read state.
`/api/notes`	6	Note listing, upload, download counting, gated file access, deletion.
`/api/qa`	14	Questions, answers, votes, accepted answers, bookmarks, follows, comments.
`/api/tutors`	13	Directory, applications, availability, bookings, session credits.
`/api/premium`	10	Paid notes, purchase flow, cancellation, receipts, entitlement checks.
`/api/payments`	3	Generic payment initiation, status polling, transaction history.
`/api/admin`	14	Statistics, user management, tutor review, paid-note review queue, payment diagnostics.
`/api/users`	5	Public profiles, statistics, leaderboard, profile updates.
`/api/notifications`	3	In-app notification list and read state.
`/api/contact`	3	Public contact form plus admin inbox.
`/api/stats`	1	Public platform counters for the landing page.

Services

Domain logic lives in `src/services/` so routes stay thin and concerns are not duplicated across endpoints.

SERVICE	WHAT IT OWNS
`entitlements.js`	Order pricing and granting. Two invariants: prices are **always** resolved server-side from the database — the client never sends an amount — and granting is idempotent, so concurrent status polls race on a conditional update and exactly one caller performs the grant.

SERVICE	WHAT IT OWNS
`mobileMoney.js`	The only module that talks to the MeSomb SDK. Phone normalisation, operator detection, asynchronous collect requests, and provider error translation.
`reconciler.js`	A background sweep that settles payments the browser stopped watching — closed tab, lost signal, late PIN entry. This is what guarantees a successful charge eventually grants access.
`paymentHealth.js`	Verifies payment credentials at startup, read-only. Turns "payments mysteriously fail at checkout" into one obvious line in the deploy log.
`fileAccess.js`	Controlled access to note files across two storage backends, and CDN redirects for free content.
`contentQuality.js`	Automated quality gate on paid uploads — extracts PDF text and measures length, page count, byte size and lexical variety.
`uploaderTrust.js`	Decides whether an uploader has earned the right to publish paid notes without review. Trust is **computed** from review history, never stored as a flag.

> **WHY TRUST IS COMPUTED, NOT STORED**
>
> A stored trust flag has to be revoked by something remembering to revoke it. A computed one drops away by itself the moment a note is rejected, because the window that produced it no longer qualifies. There is no state that can drift out of step with reality. The current rule: at least 3 approvals and zero rejections in the last 10 reviewed uploads.

6 Data model

PostgreSQL, defined entirely in `backend/prisma/schema.prisma` (476 lines, 25 tables, 7 migrations). Prisma generates the query client from this file, so the schema is the single source of truth — the database is never altered by hand.

Tables by domain

DOMAIN	TABLES	NOTES
Identity	`users`	Name, email, hashed password, university, field of study, bio, avatar, role, reputation, active flag.
Groups	`groups`, `user_groups`, `group_join_requests`, `group_read_status`	`user_groups` is the membership join table carrying the member's role. `group_read_status` drives unread badges.
Messaging	`group_messages`, `group_message_reactions`	Messages carry an optional attachment, an optional `reply_to` self-reference, and edit/delete flags. Deletes are soft, so the bubble becomes "This message was deleted" rather than vanishing mid-conversation.
Notes	`notes`	Title, description, subject, tags, file URL and type, download count, premium flag and price.
Q&A	`questions`, `answers`, `question_votes`, `answer_votes`, `question_bookmarks`, `question_followers`, `answer_comments`	Votes are separate tables rather than counters so a user cannot vote twice. Questions and answers both support audio and images.
Tutoring	`tutors`, `tutor_availability`, `bookings`, `session_credits`	`tutors` holds the application and its status. `session_credits` tracks prepaid session packs.
Commerce	`transactions`, `premium_notes`, `purchased_notes`, `premium_subscriptions`	`transactions` is the payment ledger. `purchased_notes` is the entitlement record checked before any paid file is served.

DOMAIN	TABLES	NOTES
Platform	`notifications, contact_messages`	In-app notifications and the contact-form inbox.

The two connection URLs

The datasource block declares both `url` and `directUrl`, and both are required:

```
datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")   // pooled — the running app
  directUrl = env("DIRECT_URL")     // direct  — migrations only
}
```

> **OPERATIONAL TRAP**
>
> Neon's pooled endpoint runs PgBouncer in transaction mode, which cannot carry session-level operations. Prisma migrations use advisory locks and `CREATE TYPE`, both of which are session-scoped. Running a migration through the pooled URL fails in a way that is not obviously about pooling. The pooled URL contains `-pooler` in the hostname; the direct one does not.

Working with the schema

```
# after editing schema.prisma
npx prisma migrate dev --name describe_the_change   # local: migrate + regenerate
npx prisma generate                                 # regenerate client only
npx prisma migrate deploy                           # production: apply, never create
npx prisma studio                                   # browse data in a GUI
```

On Windows, `prisma generate` fails with an `EPERM` error if the dev server is running, because it holds the query-engine DLL open. Stop the backend first.

7 Authentication & authorisation

Passwords

Hashed with bcrypt at 12 rounds (`BCRYPT_ROUNDS` , default 12). The plaintext is never stored or logged.

Login runs a bcrypt comparison even when the email does not exist, comparing against a dummy hash and discarding the result. Without this, a missing account returns measurably faster than a wrong password, which lets an attacker enumerate valid email addresses by timing alone.

Sessions

Stateless JWT signed with `JWT_SECRET` , expiring after `JWT_EXPIRES_IN` (default 24h). The token is stored in `localStorage` and attached by the Axios request interceptor. There is no server-side session store, so there is nothing to invalidate on logout beyond clearing the client — a consequence worth knowing (section 15).

Authorisation middleware

MIDDLEWARE	BEHAVIOUR
`authenticate`	Requires a valid token; sets `req.userId` . Rejects with 401 otherwise.
`optionalAuth`	Sets `req.userId` when a valid token is present but never rejects. Used where a page is public yet renders differently for a signed-in user — for instance showing whether they already own a premium note.
`requireRole(...roles)`	Factory that re-reads the role from the database and enforces it. Must run after `authenticate` . Reading from the database rather than trusting the token's claim means a demotion takes effect immediately rather than at token expiry.
`admin`	Convenience wrapper for `requireRole('admin')` .

Socket authentication

The WebSocket handshake carries the same JWT. It is verified before the connection is accepted, and the resulting user id determines which group rooms the socket may join — membership is checked against the database, not taken from the client.

8 File storage & paid-content protection

Upload pipeline

Multer parses the multipart request and streams directly to Cloudinary via `multer-storage-cloudinary` — nothing is written to the application's disk, which matters because Render's filesystem is ephemeral and would lose uploads on every redeploy.

Size limit	20 MB per file
Folder	`studyhub/`
Resource type	Chosen per file: `image` for images and PDFs, `video` for audio (Cloudinary treats audio as video), `raw` for everything else
Type check	MIME allow-list; anything else is rejected with a clear error

Free content — CDN redirect

Free notes are not proxied through the API. The server responds with a redirect to the Cloudinary URL, so the file is delivered by the CDN rather than streamed through a free-tier Node process. Before this change a 3.19 MB PDF could take the full 300-second timeout; afterwards it served in about 31 seconds on the same connection.

Paid content — entitlement gate

Premium files must never be reachable by URL alone. Two storage backends are in play and both are closed:

- **Local (legacy).** Premium files were moved into `uploads/protected/`. The static mount explicitly refuses that directory with a 403, so the only way in is a route that checks entitlement first.
- **Cloudinary (current).** Paid files are uploaded as `authenticated` delivery type, which cannot be fetched from a plain URL. The server issues a short-lived signed URL only after confirming a `purchased_notes` row exists for that user and note.

Admins bypass the purchase check by role, which is how the review queue can open a submitted file before deciding on it.

9 Payments

Payments are Mobile Money only — MTN MoMo and Orange Money — through MeSomb, a Cameroonian aggregator. There is no card processing.

The flow

1. The student enters a 9-digit local number. The client detects the operator from the prefix and selects it automatically; a mismatch between the number and the chosen service is rejected before anything is sent.
2. `POST /api/premium/pay/initiate` — the server resolves the price **from the database**, creates a `transactions` row in `pending`, and issues an **asynchronous** collect request so the HTTP call returns immediately rather than blocking for the length of the USSD interaction.
3. The provider pushes a USSD prompt to the payer's handset. The UI shows "Check your phone" and begins polling status every 3 seconds, for up to 3 minutes.
4. On success the server grants the entitlement — idempotently — and the client receives the receipt.
5. If the poll window closes without a verdict, the UI says the charge may still land and tells the user not to pay again. Settlement is left to the reconciler.

Cancellation

`POST /api/premium/pay/cancel/:txId` does not simply mark the transaction cancelled. It asks the provider first, because a payment may have succeeded in the moment between the user deciding to cancel and the request arriving:

PROVIDER SAYS	SERVER DOES
SUCCESS	Completes the order and grants access. Nobody pays and walks away empty-handed.
FAILED	Marks the transaction failed.
No reference yet	Cancels — nothing was ever charged.
PENDING	Leaves it pending and hands it to the reconciler, telling the user it is still being confirmed.

The reconciler

The browser poll is a convenience, not the source of truth. A payer can close the tab, lose signal, or approve the prompt after the page gave up. A background sweep re-checks pending transactions against the provider, completes the ones that succeeded, and abandons the ones that never will. Its intervals are tunable through `RECONCILE_INTERVAL_MS`, `RECONCILE_LOOKBACK_MS` and `RECONCILE_ABANDON_MS`.

> **HARD-WON DETAIL: NON-ASCII BREAKS THE SIGNATURE**
>
> MeSomb signs each request with an HMAC computed over the JSON body. Any character above U+007F in that body causes the provider to reject the request with "Bad signature" — an error that points at the credentials rather than at the payload. A note title containing a typographic em dash was enough to break every purchase of that note while leaving others working.
>
> The fix lives in `mobileMoney.js`: a `toAsciiSafe()` helper folds smart quotes, dashes and ellipses to their ASCII equivalents, strips anything remaining above U+007F, collapses whitespace and truncates to 120 characters. Any new field sent to the provider must pass through it.

Receipts

A receipt is available as JSON for the in-app modal and as a server-rendered HTML document at `GET /api/premium/pay/receipt/:txId?format=html`. The document is fetched with the auth header and handed to the browser as a blob, because building it in a popup and calling `print()` is blocked or mangled by most mobile browsers. If the popup is blocked, it falls back to a direct download so the receipt is never silently lost.

10 Realtime messaging

Group chat uses Socket.IO. The JWT is verified during the handshake, and each group is a room named `group:<id>`. Membership is checked server-side before a socket is allowed to join a room.

EVENT	DIRECTION	MEANING
`group:join`	client → server	Request to join a group room; membership is verified first.
`group:leave`	client → server	Leave the room.
`typing:start` `typing:stop`	client → server	Typing indicator.
`group:joined`	server → client	Join confirmed.
`message:new`	server → room	A message was posted.
`message:edit`	server → room	A message was edited (15-minute window).
`message:delete`	server → room	A message was soft-deleted.
`message:reaction`	server → room	Reactions on a message changed.
`typing:update`	server → room	Current list of typing users.
`user:joined` `user:left`	server → room	Presence changes.

Polling as a fallback, not a driver

The client also polls for messages, but at a much slower cadence when the socket is connected — 30 seconds, against 4 seconds when it is not. The poll exists to heal events missed during a brief disconnect, not to drive the chat.

Sends are optimistic: the message appears immediately with a pending marker and is replaced by the server's copy when it arrives. The socket echo is suppressed for a message the client already inserted, otherwise the sender would see their own message twice.

11 Security posture

Transport

HTTPS is enforced in production by a 308 redirect, and Helmet's HSTS header stops the browser making the insecure request a second time. Because Render terminates TLS at its proxy, `req.secure` is false even on an HTTPS request — `x-forwarded-proto` is the real signal, and it is trustworthy only because `trust proxy` is set.

Headers

Helmet applies a Content-Security-Policy restricting scripts to `'self'`, objects to `'none'` and frame ancestors to `'none'`, plus `nosniff` and a same-site cross-origin resource policy.

CORS

Origins come from `ALLOWED_ORIGINS` (comma-separated) with `FRONTEND_URL` as a fallback. Credentials are enabled, which means a wildcard is not permitted. Two deliberate diagnostics exist here, because CORS failures are otherwise very hard to trace:

- A rejected origin is logged **by name**, alongside the allow-list. A trailing slash or an `http` / `https` mismatch is invisible otherwise.
- If a production server boots with only localhost in its allow-list, it prints a loud startup error. Left unnoticed, that configuration rejects every browser request with an opaque 403 that reads like a frontend bug.

Rate limiting

SCOPE	WINDOW	MAX	RATIONALE
All `/api/`	15 min	500	General abuse ceiling.
Register, change password	15 min	20	Account-creation spam.
Login	15 min	10	Credential stuffing.
Payment initiation	1 hour	20	Money-moving endpoints.
Payment status polling	1 hour	600	Polling is high-frequency by design and must not be caught by the strict limit above.

All five are overridable by environment variable, so limits can be tightened in production without a code change.

Input and abuse handling

- **Honeypot** on public forms — an off-screen field that a human never reaches. Hidden by clip rather than `display:none`, because many bots skip fields the browser would not render.
- **Upload allow-list** by MIME type, with a 20 MB ceiling.
- **Server-side pricing.** The client never sends an amount; every price is read from the database at charge time.
- **Generic error responses.** Stack traces are logged, never returned.

12 Internationalisation

The interface is available in English and French, 1,228 keys each, covering every page including the legal documents.

Language selection

Detection order is `localStorage → navigator → htmlTag`. A stored choice deliberately beats the browser: someone who has explicitly picked a language should not have it overridden on their next visit. `nonExplicitSupportedLngs` maps `fr-CM` and `fr-FR` onto `fr` — Cameroonian browsers commonly report a region, and without it those visitors would silently get English. The document's `lang` attribute follows the choice, for screen readers and search engines.

The toggle appears in the navbar and again in Settings, since someone who has landed in the wrong language looks in Settings first.

What is translated and what is not

> **THE CENTRAL RULE**
>
> Values that are **stored or searched** stay English; only their labels are translated. Subjects, fields of study, availability days, Q&A categories and booking statuses are all written to the database. Translating the stored value would mean a French tutor's "Mathématiques" never matching an English student's "Mathematics", and the two would quietly stop finding each other.
>
> The pattern is `t(`subjectName.${s}`, { defaultValue: s })` — translate for display, fall back to the raw value for anything a user typed freely.

Other decisions worth knowing:

- **Plurals** go through i18next's `_one` / `_other` forms, not an appended "s". French says "il y a 1 mois" and "il y a 3 mois" identically, which string concatenation cannot express.
- **Dates and numbers** format in the reader's locale via `lib/formatDate.js`, which reads the active language at call time.
- **The delete-account confirmation** compares typed input against the translated word. Translating only the visible word would have left French users typing SUPPRIMER at a permanently disabled button.
- **Toast copy** reads from the i18n instance rather than the `t` hook, which keeps `t` out of fetch closures and their dependency arrays.
- **The honeypot label** stays English on purpose: it is `aria-hidden` and not a tab stop, so no person ever reads it.
- **Legal pages** carry a line stating the English version prevails in case of discrepancy.

Guarding against drift

`npm run check:i18n` compares the two locale files and fails on four things: a key present in one language but not the other, placeholder sets that differ between languages, a plural family missing its `_one` or `_other`, and a key referenced in code but defined nowhere. It also reports keys identical in both languages and keys nothing references, without failing.

It has already earned its place — it found that Login's submit button and NotFound's suggestion cards were still hardcoded English after those pages were believed finished.

13 Deployment

Frontend — Vercel

Root directory	`frontend`
Build command	`npm run build`
Output	`dist`
Rewrite	All paths → `/index.html`, so client-side routes resolve on a hard refresh
Cache headers	`index.html` and `sw.js` set to `must-revalidate` so a deploy is picked up immediately

> **BUILD-TIME, NOT RUNTIME**
>
> Vite inlines `VITE_*` variables into the bundle at **build** time. Changing one in the Vercel dashboard has no effect until the project is redeployed. A missing or wrong `VITE_API_URL` produces a frontend that loads perfectly and then fails every request — which looks like a backend outage and is not.
>
> `vercel.json` is also validated against a strict schema. Comment-style keys such as `"//"` are rejected outright and block the deployment.

FRONTEND ENVIRONMENT VARIABLES

VARIABLE	REQUIRED	PURPOSE
`VITE_API_URL`	Yes	Backend base URL including `/api`. Baked in at build time.
`VITE_SITE_URL`	Recommended	Public origin. Used to generate absolute sitemap URLs and canonical tags.
`VITE_PLAUSIBLE_DOMAIN`	Optional	Enables analytics. Omitting it disables tracking cleanly.

Backend — Render

Root directory	`backend`
Build command	`npm install && npx prisma generate`
Start command	`npm start`
Health check	`GET /health`

BACKEND ENVIRONMENT VARIABLES

VARIABLE	REQUIRED	PURPOSE
`DATABASE_URL`	Yes	Neon **pooled** connection string (hostname contains `-pooler`).
`DIRECT_URL`	Yes	Neon **direct** connection string. Migrations only.
`JWT_SECRET`	Yes	Signing secret. A long random value; rotating it invalidates every session.
`JWT_EXPIRES_IN`	No	Token lifetime. Default `24h`.
`BCRYPT_ROUNDS`	No	Password hashing cost. Default `12`.
`ALLOWED_ORIGINS`	Yes	Comma-separated frontend origins. Scheme included, no trailing slash.
`FRONTEND_URL`	Fallback	Used when `ALLOWED_ORIGINS` is unset.

`CLOUDINARY_CLOUD_NAME`
`CLOUDINARY_API_KEY`
`CLOUDINARY_API_SECRET`
	Yes	File storage credentials. Without them every upload fails with a specific, named error rather than a blank 500.

`MESOMB_APPLICATION_KEY`
`MESOMB_ACCESS_KEY`
`MESOMB_SECRET_KEY`
	Yes	Mobile Money credentials. Verified read-only at startup.

`NODE_ENV`	Yes	Must be `production` on the deployed server — it gates the HTTPS redirect.
`PORT`	No	Supplied by Render. Defaults to 5000 locally.

VARIABLE	REQUIRED	PURPOSE
`RATE_*`	No	Five window/max pairs overriding the rate limits in section 11.

`RECONCILE_INTERVAL_MS`
`RECONCILE_LOOKBACK_MS`
`RECONCILE_ABANDON_MS`
	No	Payment reconciler timing.

Database — Neon

Neon's console shows a connection string with a "pooled connection" toggle. Both forms are needed: the pooled one (toggle on, hostname contains `-pooler`) for `DATABASE_URL`, the direct one (toggle off) for `DIRECT_URL`. Choose a region close to the backend host — a database in a different continent from the API adds latency to every query.

Deploying a change

```
git push origin master        # both hosts build automatically

# if the schema changed, apply migrations against production:
cd backend
DATABASE_URL=<direct-url> npx prisma migrate deploy
```

14 Local development

Prerequisites

Node 18 or newer, npm, and access to a PostgreSQL database (a Neon branch works well and avoids installing Postgres locally).

First-time setup

```
git clone https://github.com/Tazanu/studyhubwebapp.git
cd studyhubwebapp

# ── backend ──────────────────────────────────────────
cd backend
npm install
cp .env.example .env          # then fill in the values from section 13
npx prisma migrate dev        # create the schema
npm run dev                   # http://localhost:5000

# ── frontend (new terminal) ──────────────────────────
cd ../frontend
npm install
echo "VITE_API_URL=http://localhost:5000/api" > .env
npm run dev                   # http://localhost:5173
```

Everyday commands

COMMAND	EFFECT
`npm run dev`	Start with hot reload (frontend) or auto-restart (backend).
`npm run build`	Production bundle. **Treat this as the real gate** — a mistyped Tailwind theme token fails silently in dev but breaks the build.
`npm run preview`	Serve the production build locally. Use this rather than the dev server when testing route-level behaviour.
`npm run lint`	ESLint. The baseline is 42 errors / 13 warnings, all pre-existing — a change should not increase either number.
`npm run check:i18n`	Locale parity check. Must pass before shipping a string change.
`npx prisma studio`	Browse and edit database rows in a GUI.

Common local problems

SYMPTOM	CAUSE AND FIX
`EADDRINUSE` on port 5000	A previous backend is still running. Kill the Node process and restart.
`EPERM` during `prisma generate`	The dev server holds the query-engine DLL open on Windows. Stop the backend first.
Every request fails with 403	CORS. Check `ALLOWED_ORIGINS` includes `http://localhost:5173` exactly — scheme included, no trailing slash. The server logs the rejected origin by name.
Uploads fail with a 500	`CLOUDINARY_*` is unset. The error handler returns a specific message naming the missing configuration.
Payments return "Bad signature"	Either the MeSomb keys are wrong, or a non-ASCII character reached the request body — see section 9.

15 Known limitations

Recorded honestly, because a handover that omits them just transfers the surprises.

Functional gaps

AREA	CURRENT STATE
Tutor messaging	The "Message tutor" modal writes to `localStorage` only. Nothing is sent and the tutor never receives it. It needs a real endpoint and table.
Session resources	The downloads on the tutor profile are simulated with a timer — they produce toasts but no file.
Email notifications	Not implemented. Notifications are in-app only; the Settings page says so.
Notification preferences	The toggles in Settings are component state and are not persisted.
Backend copy is English-only	Notification bodies, some payment messages and the HTML receipt are composed server-side and are not translated. Full French coverage needs backend work.
Landing-page statistics	"5,000+ active students" and "500+ peer tutors" are hardcoded placeholders, not real figures. They should be wired to `/api/stats` or removed.

Technical debt

- **No automated tests.** There is no test runner. Verification is currently lint, a successful production build, the i18n checker, and manual click-through. This is the largest single gap.
- **Lint baseline of 42 errors.** Mostly `react-hooks/set-state-in-effect` and unused variables. They are pre-existing and tracked rather than fixed; the working rule is that a change must not increase the count.
- **No token revocation.** Sessions are stateless JWTs, so a token remains valid until it expires even after logout or a ban. A short expiry limits the window; a deny-list would close it.
- **Polling alongside sockets.** Several pages poll on a timer. It is a reasonable fallback but it costs requests on a free tier.
- **Two reputation ladders.** The dashboard widget and the profile page use different tier names and thresholds, deliberately but confusingly.

Operational constraints

- **Free-tier sleep.** The backend sleeps after ~15 minutes idle; the first visitor after a quiet spell waits roughly 30–50 seconds. Mitigated but not eliminated.
- **Free-tier database expiry.** Render's free PostgreSQL expired once and took the application down with it, which is why the database now lives on Neon. Whatever the host, the expiry policy is worth knowing before it matters.
- **Single region.** One backend instance in one region. Latency outside it is unimproved.

A Appendix — API reference

All 88 endpoints. Every path is prefixed with `/api`. Unless noted, endpoints require a valid bearer token.

Authentication — `/auth`

METHOD	PATH	PURPOSE
POST	/register	Create an account. Public. Optionally submits a tutor application in the same request.
POST	/login	Exchange credentials for a token. Public.
GET	/me	Current user.
POST	/change-password	Change password, verifying the current one.

Groups — `/groups`

METHOD	PATH	PURPOSE
GET	/	List groups with membership and unread counts.
GET	/:id	Group detail including members.
POST	/	Create a group.
PATCH	/:id	Update name or description. Owner or admin.
DELETE	/:id	Delete a group. Owner or admin.
POST	/:id/join	Join, or raise a join request if the group is gated.
DELETE	/:id/leave	Leave a group.
GET	/:id/messages	Message history; accepts a `search` query.
POST	/:id/messages	Post a message, optionally with an attachment or a reply reference.
PATCH	/:id/messages/:messageId	Edit, within 15 minutes.
DELETE	/:id/messages/:messageId	Soft-delete. Author, or group admin as moderator.
POST	/:id/messages/:messageId/reactions	Toggle an emoji reaction.
POST	/:id/read	Mark the group read.
POST / GET	/:id/typing	Typing indicator fallback for when the socket is down.
GET	/:id/members	Member list with roles.
GET	/:id/requests	Pending join requests. Group admin.
POST	/:id/requests/:requestId/approve	Approve a request.
POST	/:id/requests/:requestId/deny	Deny a request.
DELETE	/:id/members/:userId	Remove a member.
GET	/:id/activity	Recent group activity.

Notes — `/notes`

METHOD	PATH	PURPOSE
GET	/	List notes. Filterable by subject and uploader.

METHOD	PATH	PURPOSE
GET	/:id	Note detail, including whether the caller owns it.
POST	/	Upload a note (multipart).
POST	/:id/download	Increment the download counter.
GET	/:id/file	Fetch the file. Redirects to the CDN for free notes; entitlement-checked for paid ones.
DELETE	/:id	Delete. Uploader or admin.

Q&A — `/qa`

METHOD	PATH	PURPOSE
GET	/	List questions. Sort, search, category, pagination.
GET	/:id	Question with answers; increments the view count.
POST	/	Ask a question, optionally with audio and images.
PATCH / DELETE	/:id	Edit or delete. Author or admin.
POST	/:id/vote	Up- or down-vote a question.
POST	/:id/bookmark	Toggle bookmark.
POST	/:id/follow	Toggle follow.
POST	/:id/answers	Post an answer.
POST	/:id/answers/:answerId/accept	Accept a best answer. Question author only.
POST	/:questionId/answers/:answerId/vote	Vote on an answer.
POST	/:questionId/answers/:answerId/comments	Comment on an answer.
GET	/user/bookmarks	The caller's bookmarks.
GET	/meta/categories	Available categories.

Tutors — `/tutors`

METHOD	PATH	PURPOSE
GET	/	Approved tutor directory. Public.
GET	/:id	Tutor profile with subjects, rate and availability.
POST	/	Submit a tutor application.
PATCH	/me	Update own tutor profile.
GET	/status/me	Caller's application status.
GET	/availability/me	Own availability slots.
POST	/:id/availability	Add a slot.
DELETE	/availability/:slotId	Remove a slot.
POST	/:id/bookings	Book a session. Consumes a prepaid credit when one exists.
GET	/bookings/mine	Bookings made by the caller.
GET	/bookings/tutor	Bookings received, as a tutor.

METHOD	PATH	PURPOSE
PATCH	/bookings/:id/status	Confirm, complete or cancel.
GET	/:id/credits	Remaining prepaid session credits.

Premium & payments — `/premium`, `/payments`

METHOD	PATH	PURPOSE
GET	/premium/notes	Paid-note catalogue, marking what the caller owns.
POST	/premium/notes	Publish a paid note. Runs the quality gate; auto-publishes for trusted uploaders.
GET	/premium/notes/publishing-status	Whether the caller may publish without review.
GET	/premium/notes/:id/access	Entitlement check.
GET	/premium/notes/:id/file	Signed download, after the entitlement check.
POST	/premium/pay/initiate	Start a purchase. Price resolved server-side.
GET	/premium/pay/status/:txId	Poll the outcome.
POST	/premium/pay/cancel/:txId	Stop waiting; asks the provider first.
GET	/premium/pay/receipt/:txId	Receipt as JSON, or HTML with `?format=html`.
GET	/premium/subscription/status	Publisher subscription state.
POST	/payments/initiate	Generic payment initiation (tutor bookings).
GET	/payments/status/:txId	Poll a generic payment.
GET	/payments/transactions	Caller's transaction history.

Admin — `/admin` (all require the admin role)

METHOD	PATH	PURPOSE
GET	/stats	Platform-wide counters.
GET	/users	Paginated user list with search.
PATCH	/users/:id/toggle	Ban or reactivate.
PATCH	/users/:id/role	Grant or remove admin.
GET	/tutors	Applications, filterable by status.
PATCH	/tutors/:id/status	Approve, reject or revoke.
PATCH	/tutors/:id/trust	Adjust publishing trust.
GET	/premium/notes	All paid notes.
GET	/premium/notes/pending	The review queue, with quality reports.
PATCH	/premium/notes/:id/review	Approve or reject, with a reason the author sees.
PATCH	/premium/notes/:id/toggle	Hide or show a note.
DELETE	/premium/notes/:id	Delete a note permanently.
GET	/premium/subscriptions	Publisher subscriptions.
GET	/payments/diagnostics	Payment credential health without exposing secret values.

Users, notifications, contact, stats

METHOD	PATH	PURPOSE
GET	/users/:id	Public profile.
GET	/users/:id/stats	Contribution counts.
GET	/users/:id/answers	Answers written by a user.
GET	/users/leaderboard/top	Top contributors by reputation.
PATCH	/users/profile	Update own profile, including avatar upload.
GET	/notifications/mine	Notifications with unread count.
PATCH	/notifications/:id/read	Mark one read.
POST	/notifications/mark-all-read	Mark all read.
POST	/contact	Submit the contact form. Public, honeypot-protected.
GET	/contact/messages	Contact inbox. Admin.
PATCH	/contact/messages/:id/read	Mark a message read. Admin.
GET	/stats	Public landing-page counters.
GET	/health	Health check. No `/api` prefix, no rate limit.